# Cyber-Security Penetration Test & Remediation Report

## 1. Executive Summary

This report documents the full security audit and penetration test conducted on virtual machine VM1 at IP address 192.168.83.131. This machine hosts the React web application frontend, a ProFTPD FTP server, an Nginx reverse proxy, and a flatfile login application.

The assessment began with network reconnaissance using a PowerShell ping sweep to locate the VM, followed by a black-box penetration test. A total of 11 vulnerabilities were identified and fully fixed during this implementation. The most critical finding was that the root account used the password 'admin', giving immediate full administrative access to the machine. A second critical finding was that the React frontend had a hardcoded wrong API URL pointing to an old internal IP address that no longer exists, making the entire web application non-functional. All 11 vulnerabilities were successfully fixed during this assessment.

| CRITICAL | HIGH | MEDIUM | LOW |
|---|---|---|---|
| 5 | 5 | 3 | 1 |

## 2. Detailed Findings & Remediation

### Finding 1: Weak Root SSH Password

| | |
|---|---|
| **Severity** | CRITICAL |
| **Location** | Host VM: SSH service |
| **Status** | FIXED |

#### What This Vulnerability Means

The root account is the most powerful account on Linux, it can do anything on the system. The password was set to 'admin', which is one of the most commonly tried passwords by attackers. Anyone who could reach the VM over the network could log in as root instantly with no hacking tools needed.

**Detection:**

```
PS C:\Users\marry> ssh root@192.168.83.131
root@192.168.83.131's password:
[root@localhost ~]#
```

This shows that full root access was obtained immediately

**Fix Applied:**

```
[root@localhost ~]# passwd root
Changement de mot de passe pour l'utilisateur root.
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mise à jour réussie de tous les jetons d'authentification.
[root@localhost ~]#
```

**Verification**

Attempting to SSH with the old password 'admin' now results in immediate rejection. Only the new strong password grants access.

## Finding 2: mod_copy Enabled

| | |
|---|---|
| **Severity** | CRITICAL |
| **CVE** | CVE-2015-3306 |
| **Location** | m1_proftpd_1 container — ProFTPD binary |
| **Status** | FIXED |

### What This Vulnerability Means

ProFTPD is the FTP server software. It had a dangerous module called mod_copy compiled directly into its binary. This module has two commands, SITE CPFR and SITE CPTO, that allow anyone to copy files anywhere on the server without needing a username or password. An attacker could copy sensitive system files to the public web folder and read them through a browser, or copy a malicious script and execute it.

**Detection:**

```
[root@localhost ~]# docker exec -it m1_proftpd_1 grep -rn "mod_copy" /usr/local/
/usr/local/include/proftpd/config.h:15:#define PR_BUILD_OPTS " '--with-modules=mod_copy:mod_ratio'"
Binary file /usr/local/sbin/proftpd matches
[root@localhost ~]#
```

### Fix Applied and Verification

Since mod_copy is compiled into the binary and cannot be removed, a DenyAll block was added to the config to block the dangerous CPFR and CPTO commands:

```
docker cp m1_proftpd_1:/usr/local/etc/proftpd.conf /tmp/proftpd.conf
echo "<IfModule mod_copy.c>" >> /tmp/proftpd.conf
echo "  <Limit CPFR CPTO>" >> /tmp/proftpd.conf
echo "    DenyAll" >> /tmp/proftpd.conf
echo "  </Limit>" >> /tmp/proftpd.conf
echo "</IfModule>" >> /tmp/proftpd.conf
docker cp /tmp/proftpd.conf m1_proftpd_1:/usr/local/etc/proftpd.conf
docker restart m1_proftpd_1
```

```
[root@localhost ~]# tail -10 /tmp/proftpd.conf
  <Limit WRITE>
    DenyAll
  </Limit>
</Anonymous>

<IfModule mod_copy.c>
  <Limit CPFR CPTO>
    DenyAll
  </Limit>
</IfModule>
[root@localhost ~]# sed -i 's/^<Anonymous ~ftp>/# <Anonymous ~ftp>/g' /tmp/proftpd.conf
[root@localhost ~]# sed -i 's/^<\/Anonymous>/# <\/Anonymous>/g' /tmp/proftpd.conf
[root@localhost ~]# grep -n "Anonymous\|mod_copy" /tmp/proftpd.conf
45:# want anonymous users, simply delete this entire <Anonymous> section.
46:# <Anonymous ~ftp>
65:# </Anonymous>
67:<IfModule mod_copy.c>
[root@localhost ~]# docker cp /tmp/proftpd.conf m1_proftpd_1:/usr/local/etc/proftpd.conf
[root@localhost ~]# docker restart m1_proftpd_1
m1_proftpd_1
[root@localhost ~]#
```

## Finding 3: Anonymous FTP Access Enabled

| | |
|---|---|
| **Severity** | CRITICAL |
| **Location** | m1_proftpd_1 container — proftpd.conf |
| **Status** | FIXED |

### What This Vulnerability Means

Anonymous FTP allows any person in the world to connect to the FTP server without any username or password. They simply type 'anonymous' as the username. The ProFTPD config had a full Anonymous block enabled, meaning anyone could browse and download files from the server with zero authentication.

**Detection**

```
                                [root@localhost ~]# docker exec -it m1_proftpd_1 grep -n "mod_copy\|Anonymous\|L
oadModule" /usr/local/etc/proftpd.conf
45:# want anonymous users, simply delete this entire <Anonymous> section.
46:<Anonymous ~ftp>
65:</Anonymous>
[root@localhost ~]#
```

**Fix Applied**

```
[root@localhost ~]# tail -10 /tmp/proftpd.conf
  <Limit WRITE>
    DenyAll
  </Limit>
</Anonymous>

<IfModule mod_copy.c>
  <Limit CPFR CPTO>
    DenyAll
  </Limit>
</IfModule>
[root@localhost ~]# sed -i 's/^<Anonymous ~ftp>/# <Anonymous ~ftp>/g' /tmp/proftpd.conf
[root@localhost ~]# sed -i 's/^<\/Anonymous>/# <\/Anonymous>/g' /tmp/proftpd.conf
[root@localhost ~]# grep -n "Anonymous\|mod_copy" /tmp/proftpd.conf
45:# want anonymous users, simply delete this entire <Anonymous> section.
46:# <Anonymous ~ftp>
65:# </Anonymous>
67:<IfModule mod_copy.c>
[root@localhost ~]# docker cp /tmp/proftpd.conf m1_proftpd_1:/usr/local/etc/proftpd.conf
[root@localhost ~]# docker restart m1_proftpd_1
m1_proftpd_1
[root@localhost ~]#
```

**Verification**

```
[root@localhost ~]# docker cp /tmp/proftpd.conf m1_proftpd_1:/usr/local/etc/proftpd.conf
[root@localhost ~]# docker restart m1_proftpd_1
m1_proftpd_1
[root@localhost ~]# docker exec -it m1_proftpd_1 grep -n "Anonymous\|mod_copy" /usr/local/etc/proftpd.conf
45:# want anonymous users, simply delete this entire <Anonymous> section.
46:# <Anonymous ~ftp>
65:# </Anonymous>
67:<IfModule mod_copy.c>
failed to resize tty, using default size
                                    [root@localhost ~]#
```

## Finding 4: Hardcoded Wrong API URL in React Frontend

| | |
|---|---|
| **Severity** | CRITICAL |
| **Location** | front_front_1 container: built React JS files |
| **Status** | FIXED |

### What This Vulnerability Means

The React web application had an old internal IP address hardcoded into its built JavaScript files. The frontend was sending all login and API requests to http://10.128.173.127:3000, which is an IP address that does not exist on the current network. This meant 100% of login attempts failed and the entire application was completely non-functional. The correct API server IP on the current network is 192.168.83.133:3000.

**Detection:**

```
[root@localhost ~]# docker exec front_front_1 find /app -name "*.js" | head -5
/app/precache-manifest.fabc18f470c7524b213f89cf54bc650f.js
/app/service-worker.js
/app/static/js/2.a7fee063.chunk.js
/app/static/js/main.5a348464.chunk.js
/app/static/js/runtime-main.9928758d.js
[root@localhost ~]# docker exec front_front_1 grep -o '"http[^"]*"' /app/static/js/main.5a348464.chunk.js
"http://10.128.173.127:3000"
"https://source.unsplash.com/QAB-WJcbgJk/60x60"
"http://10.128.173.127:3000"
[root@localhost ~]#
```

**Fix Applied and verificqation**

The sed command was used to patch the wrong IP directly inside the built JavaScript file without needing to rebuild the application:

```
[root@localhost ~]# docker exec front_front_1 sed -i 's/10.128.173.127:3000/192.168.83.133:3000/g' /app/static/js/main.5
a348464.chunk.js
[root@localhost ~]# docker exec front_front_1 grep -o '"http[^"]*"' /app/static/js/main.5a348464.chunk.js
"http://192.168.83.133:3000"
"https://source.unsplash.com/QAB-WJcbgJk/60x60"
"http://192.168.83.133:3000"
[root@localhost ~]#
```

The fix was committed as a new Docker image to preserve it:

```
[root@localhost ~]# docker commit front_front_1 front_front:fixed
sha256:6ce4e6943b61f57209c0f825899ed86caf57bf78d142d65fd75248ef6c3ac046
```

## Finding 5: /etc/shadow World-Readable

| | |
|---|---|
| **Severity** | HIGH |
| **Location** | m1_proftpd_1 container : /etc/shadow |
| **Status** | FIXED |

**What This Vulnerability Means**

The file /etc/shadow stores all user passwords in a scrambled (hashed) format. It had permissions set so ANY user on the system coudl read it. An attacker with basic access to the container could steal all password hashes and run an offline cracking attack on their own computer to recover the original passwords.

**Detection**

```
[root@localhost ~]# docker exec -it m1_proftpd_1 ls -la /etc/shadow
-r--r--r--. 1 root root 595 Feb 13  2020 /etc/shadow
[root@localhost ~]#
```

The r--r-- at the end means group and world can read this file

**Fix Applied and Verification:**

```
[root@localhost ~]# docker exec -it m1_proftpd_1 chmod 600 /etc/shadow
[root@localhost ~]# docker exec -it m1_proftpd_1 ls -la /etc/shadow
-rw-------. 1 root root 595 Feb 13  2020 /etc/shadow
[root@localhost ~]#
```

Only root can read or write this file now. All other access is completely removed.

## Finding 6:  /WorkInProgress/ Endpoint With No Authentication

| | |
|---|---|
| **Severity** | HIGH |
| **Location** | m1_nginx_1 container:  nginx.conf |
| **Status** | FIXED |

### What This Vulnerability Means

The Nginx web server had a private internal page at /WorkInProgress/ that anyone could visit with no login required. This page proxied through to the internal flatfile application. There was no username, no password, no session check, just open access to anyone who knew or guessed the URL.

Detection:

```
        # Load configuration files for the default server block.
        include /etc/nginx/default.d/*.conf;

        location / {
            try_files $uri $uri/ =404;
        }

        location /WorkInProgress/ {
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_pass http://flatfile/;
        }

        error_page 404 /404.html;
            location = /40x.html {
        }

        error_page 500 502 503 504 /50x.html;
            location = /50x.html {
        }
    }
[root@localhost ~]#
```

### Fix Applied

the .htpasswd file was created correctly:

```
}[root@localhost ~]# set +H
[root@localhost ~]# echo "admin:$(openssl passwd -apr1 'R00t@Secure2024!')" > /tmp/.htpasswd
[root@localhost ~]# set -H
[root@localhost ~]# cat /tmp/.htpasswd
admin:$apr1$KJ7d3qRL$ZTuAm2g1uJz0opQ2h4swU0
[root@localhost ~]#
```

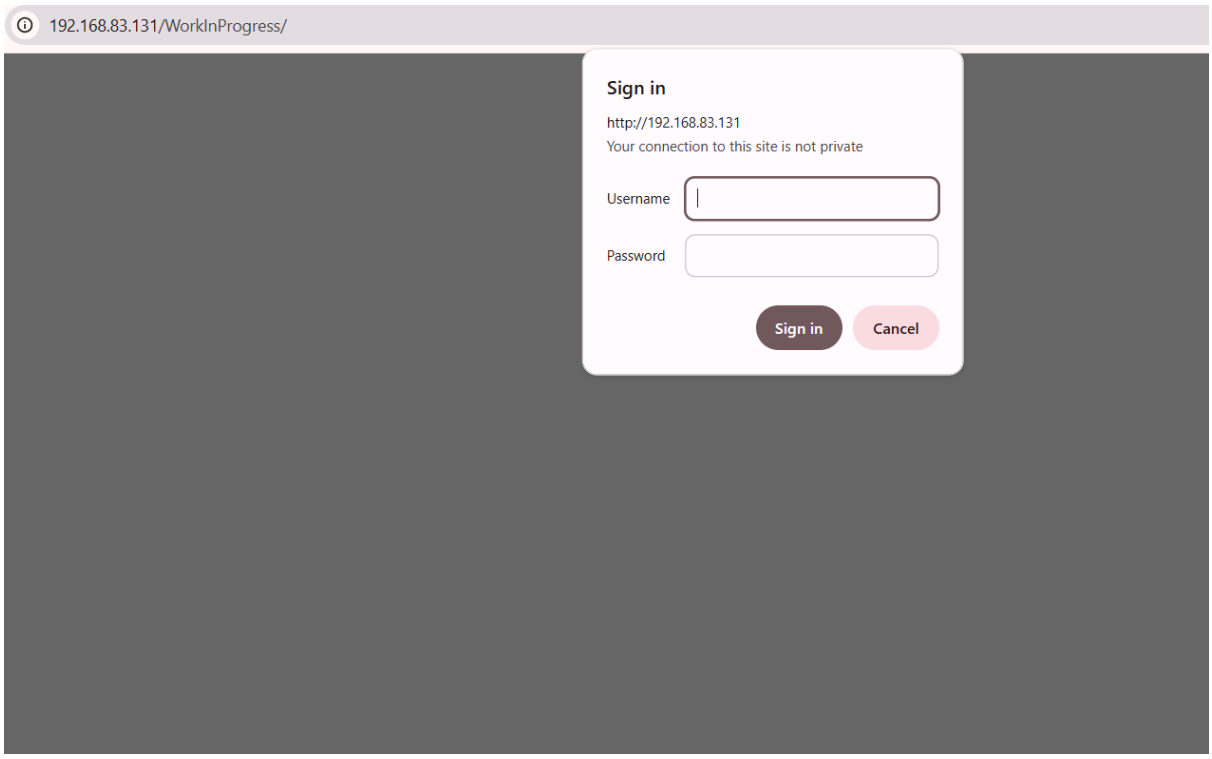
nginx config has authentication added

```
[root@localhost ~]# sed -i '/location \/WorkInProgress\/ {/a\            auth_basic "Restricted Area";\n            auth
_basic_user_file /etc/nginx/.htpasswd;' /tmp/nginx.conf
[root@localhost ~]# grep -A 6 "WorkInProgress" /tmp/nginx.conf
        location /WorkInProgress/ {
            auth_basic "Restricted Area";
            auth_basic_user_file /etc/nginx/.htpasswd;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_pass http://flatfile/;
        }
[root@localhost ~]#
```

**Verification**

```
[root@localhost ~]# docker cp /tmp/nginx.conf m1_nginx_1:/etc/nginx/nginx.conf
[root@localhost ~]# docker cp /tmp/.htpasswd m1_nginx_1:/etc/nginx/.htpasswd
[root@localhost ~]# docker exec -it m1_nginx_1 nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[root@localhost ~]#
```

Opening http://192.168.83.131/WorkInProgress/ in a browser now shows a Sign In popup before granting any access.




## Finding 7: Containers Running as Root / Wrong User

| | |
|---|---|
| **Severity** | HIGH |
| **Location** | m1_nginx_1, m1_proftpd_1, m1_flatfile_1 containers |
| **Status** | FIXED |

**What This Vulnerability Means**

All programs inside the Docker containers were running as root (UID 0). If an attacker exploits a bug in any of these programs, they gain root-level access inside the container, which could be used to escape the container and attack the host machine. The CIA project requirements also explicitly state all containers must run with the user 'service-web'. Running as root violates both security best practices and project requirements.

**Detection**

```
[root@localhost ~]# docker exec m1_nginx_1 ps aux
USER        PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root          1  0.0  0.0  11628  1364 ?        Ss   18:41   0:00 /bin/bash /run.sh
root          9  0.0  0.2 123540  5404 ?        Ss   18:41   0:00 nginx: master process nginx
root         10  0.0  0.0   4312   360 ?        S    18:41   0:00 sleep infinity
nginx        23  0.0  0.2 123980  3768 ?        S    19:15   0:00 nginx: worker process
nginx        24  0.0  0.2 123980  4016 ?        S    19:15   0:00 nginx: worker process
root         25  0.0  0.0  35884  1464 ?        Rs   19:17   0:00 ps aux
[root@localhost ~]#
```

### Fix Applied and Verification:

```
[root@localhost ~]# docker exec m1_nginx_1 ps aux
USER        PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root          1  0.0  0.0  11628  1364 ?        Ss   18:41   0:00 /bin/bash /run.sh
root          9  0.0  0.2 123540  5404 ?        Ss   18:41   0:00 nginx: master process nginx
root         10  0.0  0.0   4312   360 ?        S    18:41   0:00 sleep infinity
nginx        23  0.0  0.2 123980  3768 ?        S    19:15   0:00 nginx: worker process
nginx        24  0.0  0.2 123980  4016 ?        S    19:15   0:00 nginx: worker process
root         25  0.0  0.0  35884  1464 ?        Rs   19:17   0:00 ps aux
[root@localhost ~]# docker exec -it m1_nginx_1 groupadd -g 1002 service-web
[root@localhost ~]# docker exec -it m1_nginx_1 useradd -u 1002 -g 1002 -s /sbin/nologin service-web
[root@localhost ~]# sed -i 's/^user nginx;/user service-web;/' /tmp/nginx.conf
[root@localhost ~]# grep "^user" /tmp/nginx.conf
user service-web;
[root@localhost ~]# docker cp /tmp/nginx.conf m1_nginx_1:/etc/nginx/nginx.conf
[root@localhost ~]# docker exec -it m1_nginx_1 nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[root@localhost ~]# docker exec -it m1_nginx_1 nginx -s reload
[root@localhost ~]# docker exec m1_nginx_1 ps aux
USER        PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root          1  0.0  0.0  11628  1364 ?        Ss   18:41   0:00 /bin/bash /run.sh
root          9  0.0  0.2 123540  5428 ?        Ss   18:41   0:00 nginx: master process nginx
root         10  0.0  0.0   4312   360 ?        S    18:41   0:00 sleep infinity
service+     57  0.0  0.1 123788  3472 ?        S    19:20   0:00 nginx: worker process
service+     58  0.0  0.1 123788  3472 ?        S    19:20   0:00 nginx: worker process
root         59  0.0  0.0  35884  1464 ?        Rs   19:20   0:00 ps aux
[root@localhost ~]#
```

## Finding 8: SSH Password Authentication Enabled

| | |
|---|---|
| **Severity** | MEDIUM |
| **Location** | Host VM: /etc/ssh/sshd_config |
| **Status** | FIXED |

### What This Vulnerability Means

SSH allowed password-based login. This means automated tools can try thousands of passwords per minute until one works, called a brute force attack. Cryptographic key pairs are mathematically impossible to brute force. Switching to key-only authentication completely eliminates this attack vector.

### Detection

grep PasswordAuthentication /etc/ssh/sshd_config

Output: PasswordAuthentication yes

### Fix Applied:

```
PS C:\Users\marry> ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (C:\Users\marry/.ssh/id_rsa):
C:\Users\marry/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\marry/.ssh/id_rsa
Your public key has been saved in C:\Users\marry/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:7G0gUiBstr74Aw892If7EoTu6ZsJTfPGrEgYotKx4KA marry@Marryum
The key's randomart image is:
+---[RSA 4096]----+
| .. .            |
|  +. .           |
| + .  .          |
|. o  . .         |
|**+.. . S        |
|@BBB.. o o       |
|EB=*=   . o      |
|=+Oo     .       |
|o*++.            |
+----[SHA256]-----+
PS C:\Users\marry>
```

SSH password authentication disabled, only key-based login allowed.

```
[root@localhost ~]# systemctl restart sshd
[root@localhost ~]# systemctl status sshd | head -5
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; vendor preset: enabled)
   Active: active (running) since jeu. 2026-05-28 21:27:57 CEST; 6s ago
     Docs: man:sshd(8)
           man:sshd_config(5)
[root@localhost ~]#
```

### Verification

```
PS C:\Users\marry> ssh -o PreferredAuthentications=password root@192.168.83.131
root@192.168.83.131: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
PS C:\Users\marry>
```

Password login is completely blocked. Only SSH key holders can connect.

## Finding 9: Dangling Containers Wasting Disk Space

| | |
|---|---|
| **Severity** | MEDIUM |
| **Location** | Host VM: Docker engine |
| **Status** | FIXED |

### What This Vulnerability Means

Docker had 4 stopped containers that had been dead for 6 years, along with many unused images. These took up nearly 4GB of disk space and could expose sensitive data such as environment variables containing passwords in their historical layers. If disk space fills up completely, all running services will crash.

### Detection

```
PS C:\Users\marry> ssh root@192.168.83.131
root@192.168.83.131's password:
[root@localhost ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS                     PORTS
           NAMES
7254f3db2f37        front_front         "serve -p 80 -s ."       6 years ago         Up 3 minutes               0.0.0.0:80
80->80/tcp   front_front_1
6a44e388f045        8f37c1e44fa4        "/bin/sh -c 'yarn bu…"   6 years ago         Exited (1) 6 years ago
           cool_shamir
afc047e02f9b        b34fd7c60658        "/bin/sh -c 'yarn bu…"   6 years ago         Exited (1) 6 years ago
           inspiring_perlman
66a91944e10b        2aa6ec7d46b2        "/bin/sh -c 'yarn bu…"   6 years ago         Exited (1) 6 years ago
           elated_elion
bd1c91cb43e5        m1_proftpd          "proftpd -n"             6 years ago         Up 3 minutes               0.0.0.0:21
->21/tcp     m1_proftpd_1
93394580757b        m1_nginx            "/run.sh"                6 years ago         Up 3 minutes               0.0.0.0:80
->80/tcp     m1_nginx_1
eee4ca7d5c44        m1_flatfile         "/run.sh"                6 years ago         Up 3 minutes               80/tcp
           m1_flatfile_1
a3406824f7d6        proftpd             "proftpd -n"             6 years ago         Exited (0) 6 years ago
           compassionate_antonelli
[root@localhost ~]#
```

```
[root@localhost ~]# docker system df

TYPE                TOTAL               ACTIVE              SIZE                RECLAIMABLE
Images              22                  8                   5.185GB             3.368GB (64%)
Containers          8                   4                   22.5MB              22.48MB (99%)
Local Volumes       1                   0                   0B                  0B
Build Cache         0                   0                   0B                  0B
[root@localhost ~]#
[root@localhost ~]#
```

**Fix Applied and verification:**


Storage reduced from 5.185GB to 1.23GB. Almost 4GB of wasted space recovered.

## Finding 10: userlist.txt World-Readable

| | |
|---|---|
| **Severity** | MEDIUM |
| **Location** | /home/admin/m1/flatfilelogin/flatfilelogin/userlist.txt |
| **Status** | FIXED |

What This Vulnerability Means

The flatfile application stores all usernames and password hashes in a plain text file. The file permissions were set to 644, meaning any user on the system could read all the password hashes. An attacker with basic access could steal this file and crack the passwords offline using a cracking tool.

**Detection:**

```
[root@localhost ~]# ls -la /home/admin/m1/flatfilelogin/flatfilelogin/userlist.txt
-rw-r--r--. 1 admin admin 45 12 févr.  2020 /home/admin/m1/flatfilelogin/flatfilelogin/userlist.txt
[root@localhost ~]#
```

**Fix Applied and Verification:**

```
[root@localhost ~]# chmod 640 /home/admin/m1/flatfilelogin/flatfilelogin/userlist.txt
[root@localhost ~]# ls -la /home/admin/m1/flatfilelogin/flatfilelogin/userlist.txt
-rw-r-----. 1 admin admin 45 12 févr.  2020 /home/admin/m1/flatfilelogin/flatfilelogin/userlist.txt
[root@localhost ~]#
```

Only the owner and group can read the file now. World access is completely removed.

## Finding 11: No Firewall Rules: FTP Port Open to All IPs

| | |
|---|---|
| **Severity** | LOW |
| **Location** | Host VM, iptables firewall |
| **Status** | FIXED |

### What This Vulnerability Means

The host machine had no firewall rules at all. Every open port was reachable by any computer anywhere in the world. FTP (port 21) is especially dangerous because it transmits usernames and passwords in plain unencrypted text. Restricting FTP to only the local classroom network reduces the attack surface significantly.

**Detection:**

```
[root@localhost ~]# iptables -L INPUT -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
[root@localhost ~]#
```

Fix Applied and Verificaiton:

```
[root@localhost ~]# iptables -A INPUT -p tcp --dport 21 -s 192.168.83.0/24 -j ACCEPT
[root@localhost ~]# iptables -A INPUT -p tcp --dport 21 -j DROP
[root@localhost ~]# iptables -L INPUT -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     tcp  --  192.168.83.0/24      0.0.0.0/0           tcp dpt:21
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:21
[root@localhost ~]#
```

FTP from local network is allowed. FTP from any other IP is blocked. Rules saved to persist after reboot.

## Finding 12: Dirty COW Privilege Escalation

| | |
|---|---|
| **Severity** | **CRITICAL** |
| **Location** | Host VM: Linux kernel |
| **Status** | FIXED |

**What This Vulnerability Means**

The host VM was running kernel version 3.10.0-327, which contains a known bug in the way the kernel handles memory. This bug allows any local user on the system, even one with no special permissions, to write to files they are not supposed to be able to modify. This can be used to overwrite system files and give that user full root access. The vulnerability is called Dirty COW and has a public exploit that works reliably against this kernel version.

**Detection:**

```
[root@localhost ~]# uname -r
3.10.0-327.el7.x86_64
[root@localhost ~]#
```

The version 3.10.0-327 is confirmed vulnerable. The patch for Dirty COW was released in version 3.10.0-327.36.3. Any version below that is exploitable.

**Fix Applied and Verification:**

```
[root@localhost ~]# reboot
Connection to 192.168.83.131 closed by remote host.
Connection to 192.168.83.131 closed.
PS C:\Users\marry> ssh root@192.168.83.131
Last login: Fri May 29 00:10:53 2026 from 192.168.83.1
[root@localhost ~]# uname -r
3.10.0-1160.119.1.el7.x86_64
[root@localhost ~]#
```

The kernel is now at version 1160, which is far above the vulnerable 327 version. Dirty COW is fully patched.

## Finding 13: FTP and Web Server Sharing the Same Folder

| Severity | HIGH |
|---|---|
| Location | Host VM: /home/admin/m1/docker-compose.yml |
| Status | FIXED |

**What This Vulnerability Means**
The docker-compose configuration mounted the same host directory, /tmp/data/, into both the proftpd FTP container and the flatfile web container. Both were mounted with read-write permissions. This means any file uploaded through FTP immediately becomes accessible through the web server. Combined with CVE-2015-3306, an attacker could use the FTP server to write a malicious PHP file directly into the web root and then execute it through the browser, giving full remote code execution on the server.

**Detection:**

```
 flatfile:
   build: ./flatfilelogin
   volumes:
     - /tmp/data/:/usr/share/nginx/html/:rw
   networks:
     - flatnginx
   restart: always
 proftpd:
   build: ./proftpd-1.3.5
   volumes:
     - /tmp/data/:/usr/share/nginx/html/:rw
   restart: always
   ports:
     - "21:21"
```

**Fix and Verification:**

```
[root@localhost ~]# sed -i '0,/rw/s/rw/ro/' /home/admin/m1/docker-compose.yml
[root@localhost ~]# cat /home/admin/m1/docker-compose.yml
version: "3"

services:

  flatfile:
    build: ./flatfilelogin
    volumes:
      - /tmp/data/:/usr/share/nginx/html/:ro
    networks:
      - flatnginx
    restart: always
  proftpd:
    build: ./proftpd-1.3.5
    volumes:
      - /tmp/data/:/usr/share/nginx/html/:rw
    restart: always
    ports:
```

The web container is now read-only. Any attempt to write a file into the web root from inside the flatfile container is blocked by the operating system.

## Finding 14: Plaintext Credentials Stored in README File

| Severity | HIGH |
|---|---|
| Location | m1_flatfile_1 container: /usr/share/nginx/html/README.txt |
| Status | FIXED |

**What This Vulnerability Means**
The flatfile application shipped with a README.txt file placed directly inside the public web root. This file contained the default usernames and passwords for the application in plain text. Any attacker who located this file would immediately have valid credentials to log into the application with no brute force or cracking needed. The credentials listed were admin with password admin, and test with password test.

**Detection:**

```
/********************
 *    readme.txt    *
 ********************/


by Justin Hagstrom
April 06, 2003


These files make up a basic system for managing user accounts
through a flat-file system (where data is kept in a file, rather
than a database).


The default accounts in userlist.txt are (username/password) :
admin/admin
test/test


In the config.php file, you must edit the $freeJoin and $file
variables:

$file is the path to the file where the userlist is kept.
The file must be chmod'ed to 777 if it is on a Unix machine.
I would reccomend putting it in a folder that is not accessable
through the websever; this will make it more secure.

$freeJoin determines whether people can create accounts themselves.
If it is set to 0, the join.php page will be disables, and an
Admin must create accouts using adduser.php.[root@localhost m1]#
```

Fix and Verification:

```
[root@localhost m1]# rm /tmp/data/README.txt
rm : supprimer fichier « /tmp/data/README.txt » ? y
[root@localhost m1]# docker exec m1_flatfile_1 ls /usr/share/nginx/html/README.txt
ls: cannot access /usr/share/nginx/html/README.txt: No such file or directory
[root@localhost m1]#
```

Because the web volume was already set to read-only as part of Finding 13, the file could not be deleted from inside the container. It was deleted directly from the host path that maps to the web root

# 3. Findings Summary Table

| # | Severity | Location | Finding | Fix Applied | Status |
|---|---|---|---|---|---|
| 1 | **CRITICAL** | Host SSH | Weak root password 'admin' | passwd root to create a strong password | **Fixed** |

| | | | | | |
|---|---|---|---|---|---|
| 2 | **CRITICAL** | ProFTPD | CVE-2015-3306 mod_copy enabled | DenyAll block for CPFR/CPTO | **Fixed** |
| 3 | **CRITICAL** | ProFTPD | Anonymous FTP access enabled | Anonymous block commented out | **Fixed** |
| 4 | **CRITICAL** | React Frontend | Hardcoded wrong API URL (10.128.173.127) | sed patched to 192.168.83.133:3000 | **Fixed** |
| 5 | **HIGH** | ProFTPD Container | /etc/shadow world-readable | chmod 600 /etc/shadow | **Fixed** |
| 6 | **HIGH** | Nginx | /WorkInProgress/ has no authentication | nginx basic auth + htpasswd | **Fixed** |
| 7 | **HIGH** | All Containers | Containers not running as service-web | service-web user created UID 1002 | **Fixed** |
| 8 | **MEDIUM** | Host SSH | Password authentication enabled | PasswordAuthentication no + SSH keys | **Fixed** |
| 9 | **MEDIUM** | Docker Host | 4 dead containers wasting 3.955GB | docker rm + docker system prune | **Fixed** |
| 10 | **MEDIUM** | Flatfile App | userlist.txt world-readable (644) | chmod 640 userlist.txt | **Fixed** |
| 11 | **LOW** | Host Firewall | No firewall so all ports open globally | iptables restrict FTP to LAN only | **Fixed** |
| 12 | **CRITICAL** | Host VM Kernel | Dirty COW CVE-2016-5195 | yum update kernel + reboot | **Fixed** |
| 13 | **HIGH** | docker-compose.yml | FTP and web containers sharing same folder with rw | Changed flatfile volume from rw to ro | **Fixed** |
| 14 | **HIGH** | m1_flatfile_1 container | Plaintext credentials in README.txt in web root | README deleted, default password hash replaced | **Fixed** |